Core Algorithms & DSA — Generic Reference Sheet

Language-agnostic logic | Works in C++, Java, JavaScript, Python

SECTION 1 — Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?	Use when
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Never in production. Teaching only.
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No	Small arrays, minimizing swaps.
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Small or nearly sorted arrays.
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Need guaranteed performance + stability.
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No	General purpose. Fastest in practice.
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Need O(1) space + guaranteed performance.
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes	Integers in known small range.

Q: Bubble Sort — how does it work?

➜ Compare adjacent pairs. Swap if out of order. Repeat until no swaps needed. Largest element 'bubbles' to end each pass.

```
for i = 0 to n-1:
for j = 0 to n-i-2:
if arr[j] > arr[j+1]:
swap(arr[j], arr[j+1])
```

■ *O(n) best case when array already sorted — add a 'swapped' flag to detect this.*

Q: Selection Sort — how does it work?

➜ Find the minimum element in unsorted part. Swap it to the front. Repeat.

```
for i = 0 to n-1:
minIdx = i
for j = i+1 to n-1:
if arr[j] < arr[minIdx]: minIdx = j
swap(arr[i], arr[minIdx])
```

Q: Insertion Sort — how does it work?

➜ Take each element and insert it into its correct position in the already-sorted left portion.

```
for i = 1 to n-1:
key = arr[i]
j = i - 1
while j >= 0 and arr[j] > key:
arr[j+1] = arr[j]
j = j - 1
arr[j+1] = key
```

Q: Merge Sort — how does it work?

➜ Divide array into two halves. Recursively sort each half. Merge the two sorted halves. Divide and Conquer.

```
function mergeSort(arr):
if length(arr) <= 1: return arr
mid = length(arr) / 2
left = mergeSort(arr[0..mid])
right = mergeSort(arr[mid..end])
return merge(left, right)
function merge(left, right):
result = []
while left and right not empty:
if left[0] <= right[0]: append left[0] to result, remove from left
else: append right[0] to result, remove from right
append remaining elements
return result
```

■ *Guaranteed O(n log n). Uses extra O(n) space. Stable sort.*

Q: Quick Sort — how does it work?

➜ Pick a pivot. Partition: all elements smaller go left, larger go right. Recursively sort both sides.

```
function quickSort(arr, low, high):
if low < high:
pivot = partition(arr, low, high)
quickSort(arr, low, pivot-1)
quickSort(arr, pivot+1, high)
function partition(arr, low, high):
pivot = arr[high] // pick last as pivot
i = low - 1
for j = low to high-1:
if arr[j] <= pivot:
i++
swap(arr[i], arr[j])
swap(arr[i+1], arr[high])
return i+1
```

■ *Worst case $O(n^2)$ when array already sorted and pivot is always min/max. Fix: random pivot.*

SECTION 2 — Searching Algorithms

Algorithm	Time	Space	Requirement	Use when
Linear Search	O(n)	O(1)	None	Unsorted array, small data
Binary Search	O(log n)	O(1)	Array MUST be sorted	Sorted array, large data
BFS	O(V+E)	O(V)	Graph/Tree	Shortest path, level order
DFS	O(V+E)	O(V)	Graph/Tree	Cycle detection, paths, topological sort

Q: Linear Search

➜ Check every element one by one until found or end of array.

```
function linearSearch(arr, target):
for i = 0 to n-1:
if arr[i] == target: return i
return -1
```

Q: Binary Search — MUST know this cold

➜ Compare target with middle element. If equal → found. If less → search left half. If greater → search right half. Repeat.

```
function binarySearch(arr, target):
low = 0, high = n-1
while low <= high:
mid = (low + high) / 2
if arr[mid] == target: return mid
else if arr[mid] < target: low = mid + 1
else: high = mid - 1
return -1
```

■ *Array MUST be sorted first. If not sorted, sort first O(n log n) then search O(log n).*

Q: BFS — Breadth First Search

➜ Explore level by level using a QUEUE. Visit all neighbors before going deeper. Finds SHORTEST path in unweighted graph.

```
function BFS(graph, start):
queue = [start]
visited = {start}
while queue not empty:
node = queue.dequeue() // remove from front
process(node)
for each neighbor of node:
if neighbor not in visited:
visited.add(neighbor)
queue.enqueue(neighbor)
```

■ *Use BFS for: shortest path, level order traversal, connected components.*

Q: DFS — Depth First Search

➜ Go as deep as possible using a STACK (or recursion). Backtrack when dead end. Used for cycle detection, paths.

```
function DFS(graph, node, visited):
visited.add(node)
process(node)
for each neighbor of node:
if neighbor not in visited:
DFS(graph, neighbor, visited)
// Iterative version using explicit stack:
function DFS_iter(graph, start):
stack = [start], visited = {}
while stack not empty:
node = stack.pop()
if node not visited:
visited.add(node)
push all neighbors to stack
```

SECTION 3 — Core Data Structures

Structure	Access	Search	Insert	Delete	Use for
Array	O(1)	O(n)	O(n)	O(n)	Index-based access, fixed size
Linked List	O(n)	O(n)	O(1)	O(1)	Frequent insert/delete, unknown size
Stack (LIFO)	O(n)	O(n)	O(1)	O(1)	Undo, brackets, call stack
Queue (FIFO)	O(n)	O(n)	O(1)	O(1)	BFS, scheduling, print queue
Hash Map	O(1)	O(1)	O(1)	O(1)	Fast lookup, frequency count
Binary Tree	O(n)	O(n)	O(n)	O(n)	Hierarchical data
BST	O(log n)	O(log n)	O(log n)	O(log n)	Sorted data, range queries
Heap (min/max)	O(n)	O(n)	O(log n)	O(log n)	Priority queue, top-K problems

Q: Stack — LIFO. Last In First Out.

➜ Push adds to top. Pop removes from top. Peek looks at top without removing.

```
Stack operations:
push(x) → add x to top
pop() → remove and return top
peek() → return top without removing
isEmpty() → return true if empty
Use cases: undo operations, valid parenthesis, function call stack, DFS
```

Q: Queue — FIFO. First In First Out.

➜ Enqueue adds to back. Dequeue removes from front.

```
Queue operations:
enqueue(x) → add x to back
dequeue() → remove and return front
peek() → return front without removing
Use cases: BFS, print queue, task scheduling, sliding window
```

Q: Linked List — structure and traversal.

➜ Each node has data and a pointer to next node. No random access — must traverse from head.

```
Node structure:
node.data = value
node.next = pointer to next node (null if last)
Traversal:
current = head
while current != null:
process(current.data)
current = current.next
Insert at head: newNode.next = head; head = newNode
Insert at tail: traverse to last; last.next = newNode
Delete a node: prev.next = node.next
```

Q: Binary Search Tree (BST) — insert and search.

➜ Left child < parent < right child. Inorder traversal gives sorted output.

```
Insert:
if tree empty: root = newNode
if value < current: go left
if value > current: go right
repeat until empty spot found
Search:
if current == null: not found
if target == current.val: found
if target < current.val: search left
if target > current.val: search right
Inorder traversal (sorted output):
inorder(left) → process(node) → inorder(right)
```

Q: Hash Map / Hash Table — how it works internally.

➜ Key is passed through a hash function → gives index in an array. Collision handled by chaining (linked list at index) or open addressing.

```
hash(key) → index
store value at arr[index]
Collision (two keys → same index):
Chaining: store linked list at each index
Open addressing: find next empty slot
O(1) average case — O(n) worst case when all keys collide
```

■ *Always mention collision handling — they ask this.*

SECTION 4 — Algorithm Patterns (Generic Logic)

Q: Two Pointer Pattern

➜ Use two pointers moving toward each other or in the same direction. Works on sorted arrays or strings.

```
// Pattern 1: from both ends (pair sum, palindrome)
left = 0, right = n-1
while left < right:
if condition: return or record result
else if need bigger: left++
else: right--
// Pattern 2: same direction (sliding window variant)
slow = 0, fast = 0
while fast < n:
// fast explores, slow marks window start
fast++
```

Q: Sliding Window Pattern

➜ Maintain a window of elements. Expand right pointer. Shrink left pointer when condition violated. Never reset to zero — slide.

```
left = 0, maxLen = 0
for right = 0 to n-1:
add arr[right] to window
while window violates condition:
remove arr[left] from window
left++
maxLen = max(maxLen, right - left + 1)
Used for: longest substring, max sum subarray, minimum window
```

Q: Divide and Conquer Pattern

➜ Break problem into smaller subproblems. Solve each independently. Combine results. Merge Sort and Quick Sort are classic examples.

```
function solve(problem):
if base case: return simple answer
left = solve(left half)
right = solve(right half)
return combine(left, right)
```

Q: Dynamic Programming (DP) Pattern

➜ Break into overlapping subproblems. Store results to avoid recomputation (memoization or tabulation).

```
// Fibonacci — classic DP
// Without DP: O(2^n) — recalculates same values
// With DP: O(n) — store results
// Tabulation (bottom-up):
dp[0] = 0, dp[1] = 1
for i = 2 to n:
dp[i] = dp[i-1] + dp[i-2]
return dp[n]
// Ask yourself: can this problem be broken into smaller same-type problems?
// If yes → try DP
```

■ *DP = recursion + storing results. If you see 'optimal', 'maximum', 'minimum', 'count ways' → think DP.*

Q: Greedy Pattern

➜ Make the locally optimal choice at each step. Hope it leads to global optimum. Works when local optimal = global optimal.

```
// Example: Activity Selection (book max non-overlapping meetings)
Sort activities by end time
lastEnd = -infinity
for each activity:
if activity.start >= lastEnd:
select this activity
lastEnd = activity.end
// Greedy works here because finishing earliest = more room for others
```

■ *Greedy does NOT always work. Always verify: does local best = global best for this problem?*

Q: Recursion Pattern

➜ Function calls itself with a smaller version of the problem. MUST have a base case or it runs forever.

```
function recursive(input):
// BASE CASE — always first
if input is smallest possible: return answer
// RECURSIVE CASE
smaller = make input smaller (n-1, n/2, etc.)
result = recursive(smaller)
return combine current with result
// Factorial example:
factorial(n):
if n == 0: return 1 // base case
return n * factorial(n-1) // recursive case
```

SECTION 5 — String Problems — Generic Logic

Q: Reverse a string

➜ Two pointer from both ends. Swap characters moving inward.

```
left = 0, right = n-1
while left < right:
swap(str[left], str[right])
left++, right--
```

Q: Reverse words in a sentence ('my name is Faizan' → 'Faizan is name my')

➜ Split by space → reverse the array of words → join back.

```
words = split(sentence, ' ')
reverse(words array)
result = join(words, ' ')
OR: reverse entire string, then reverse each word individually
```

Q: Move last word to front ('my name is Faizan' → 'Faizan my name is')

➜ Split by space. Remove last element. Insert at beginning. Join.

```
words = split(sentence, ' ')
lastWord = words[words.length - 1]
remove last from words
insert lastWord at index 0
result = join(words, ' ')
```

Q: Check palindrome

➜ Two pointer from both ends. If any mismatch → not palindrome.

```
clean string (lowercase, remove non-alphanumeric)
left = 0, right = n-1
while left < right:
if str[left] != str[right]: return false
left++, right--
return true
```

Q: Check anagram ('listen' and 'silent')

➜ Sort both strings. If equal → anagram. OR use frequency map.

```
// Method 1: Sort
return sort(s) == sort(t)
// Method 2: Frequency map
if length(s) != length(t): return false
create freq map for s
for each char in t:
decrement freq[char]
if freq[char] < 0: return false
return true
```

Q: Find duplicate characters in a string

➜ Build frequency map. Report any character with count > 1.

```
freq = {}
for each char in string:
freq[char] = freq[char] + 1 (or 1 if not exists)
for each key in freq:
if freq[key] > 1: print key, ' is duplicate'
```

Q: Two Sum — find two numbers that add to target

➜ Use hash map. Store each number. For each element, check if complement (target - element) already in map.

```
map = {}
for i = 0 to n-1:
complement = target - arr[i]
if complement in map:
return [map[complement], i]
map[arr[i]] = i
return not found
```

■ *O(n) — better than brute force O(n²) nested loops.*

Q: Find 2nd highest number

➜ Single pass with two variables. Track highest and second highest separately.

```
first = -infinity
second = -infinity
for each num in array:
if num > first:
second = first
first = num
else if num > second AND num != first:
second = num
if second == -infinity: return null (no distinct 2nd)
return second
```

■ *Handle edge case: [5,5,5] has no distinct 2nd highest → return null/–1.*

Q: Valid Parenthesis — stack problem

➜ Push opening brackets. When closing bracket found, pop and check if it matches.

```
stack = empty
for each char in string:
if char is opening bracket ( [ { :
push to stack
else if char is closing bracket ) ] } :
if stack empty OR top of stack doesn't match:
return false
pop from stack
return stack is empty
```

SECTION 6 — Quick Complexity Reference

Operation / Algorithm	Time Complexity	Space	Key Point
Access array by index	O(1)	O(1)	Direct memory address
Search unsorted array	O(n)	O(1)	Must check every element
Binary Search	O(log n)	O(1)	Array must be sorted
Bubble / Selection / Insertion Sort	$O(n^2)$	O(1)	Simple but slow
Merge Sort	O(n log n)	O(n)	Stable, guaranteed
Quick Sort	O(n log n)	O(log n)	Fast in practice, $O(n^2)$ worst
Hash Map get/put	O(1)	O(n)	O(n) worst case collision
BST search/insert	O(log n)	O(n)	O(n) worst if unbalanced
BFS / DFS	O(V + E)	O(V)	V=vertices, E=edges
Heap insert/delete	O(log n)	O(n)	Used for priority queue
Reverse string/array	O(n)	O(1)	Two pointer in-place
Check palindrome	O(n)	O(1)	Two pointer from ends
Anagram check (sort)	O(n log n)	O(n)	Sort both, compare
Two Sum (hash map)	O(n)	O(n)	Better than $O(n^2)$ brute force

State brute force first. Then say 'we can do better'. Then give optimal. Always say time and space complexity out loud.